Lecture 4
Wildcards & Redirection

Comp 206 – Introduction to Software Systems – Fall 2026

Mathieu Bérubé-Vallières

McGill University

September 8, 2026

Commands (review and more)

- Command syntax
  - command -[options] argument1 argument2 …
- Command-line prompt:
  - username@hostname:currentdirectory$ [enter command]
- man: **man**ual pages for linux commands
  - Example: `man ls`
  - An exception is built-in shell commands, use `man builtins` to see the list

Bérubé-Vallières © 2026
See last slide for complete credits

Commands (review and more)

- Path
  - Absolute path (starts at root) vs relative path (starts where you are)
    - Directory separator in Unix (Linux, MacOS): forward slash /
      - (unlike Windows, which primarily uses backward slash \)
  - Special symbols: ~ (home), / (root), . (current), .. (parent)
    - ! On the mimi server, there is a directory called `/home` where all the users' homes are located. This is different than *your* home, which is referred to as ~ and is located somewhere like `/home/2018/mberub20`. This is the directory you are automatically in when logging in
    - Some of you also noticed that there are some places where you cannot write: for example, at /. We will learn more about permissions next class
- pwd: **p**rint **w**orking **d**irectory
  - Example: `pwd`
- cd: **c**hange **d**irectory
  - Examples: `cd Docs, cd /home/user/Docs`

Bérubé-Vallières © 2026
See last slide for complete credits

Commands (review and more)

- ls: **ls**t directories
  - Examples: `ls`, `ls file1 file2`
  - `ls -l`: see long listing format (i.e. detailed information)
  - `ls -a`: see all files (i.e. see hidden files, which start with `.`)
  - Without arguments, lists files in current directory, otherwise list specified files or the list of files inside specified directories
- Flags can be combined
  - True in general for all commands
  - Example: `ls -la` is equivalent to `ls -l -a` and will show the long listing format for all files including hidden ones
- We saw `touch`, `cat` and `more`
  - Basic commands to create and read files. We will learn how to create and edit files using the vim text editor instead next class
  - `less`: updated version of `more`

Bérubé-Vallières © 2026
See last slide for complete credits

Commands (review and more)

- mkdir: **ma**k**e** **dir**ectories
- cp: **co**p**y** files
  - Usage 1: `cp [options] source destination`
  - Usage 2: `cp [options] source1 source2 … directory`
  - Examples: `cp ex.txt Docs/ex.txt` (Usage 1) `cp ex.txt Docs` (Usage 2)
  - If the target is a directory, this is usage 2 and the file(s) will be copied inside of it. Otherwise, this is usage 1 and the file will be copied as destination
- mv: **mo**v**e** (rename) files
  - Usage 1: `mv [options] source destination`
  - Usage 2: `mv [options] source1 source2 … directory`
  - If the target is a directory, this is usage 2 and the file(s) will be moved inside of it. Otherwise, this is usage 1 and the file will be renamed as destination

Commands (review and m

Remember that `.` and `..` can be used, for example
`cp ../ex.txt .`
copies the `ex.txt` file from the parent directory to the current directory

- mkdir: **mak**e **dir**ectories
- cp: **co**p**y** files
  - Usage 1: `cp [options] source destination`
  - Usage 2: `cp [options] source1 source2 … directory`
  - Examples: `cp ex.txt Docs/ex.txt` (Usage 1) `cp ex.txt Docs` (Usage 2)
  - If the target is a directory, this is usage 2 and the file(s) will be copied inside of it. Otherwise, this is usage 1 and the file will be copied as destination
- mv: **mo**v**e** (rename) files
  - Usage 1: `mv [options] source destination`
  - Usage 2: `mv [options] source1 source2 … directory`
  - If the target is a directory, this is usage 2 and the file(s) will be moved inside of it. Otherwise, this is usage 1 and the file will be renamed as destination

Commands (review and more)

- rm: **r**emo**v**e files
  - Usage: `rm [options] file1 file2 …`
  - Removes for good! There is no way to recover files. (This is also true when overwriting files with cp and mv)
- Options for cp, mv, and rm:
  - -i : interactive (cp, mv and rm): prompt and wait for confirmation before proceeding
  - **-r or -R: recursive (cp, rm): recursively visits a directory, first visiting the files and subdirectories beneath it – This is how you copy or remove directories**
  - -f : force : (mv, rm): don't prompt for confirmation (overrides -i if placed after)

Bérubé-Vallières © 2026
See last slide for complete credits

Commands (review and more)

- When pressing the up/down arrow keys, you will go through the history of the commands you typed
- You can also see a list by typing `history`, or search through previous commands with Ctrl+R
  - (History keeps 500 commands by default)
- Can also autocomplete with Tab
- You can exit a shell by using `exit` (can also sign-out of login shell, e.g. mimi, using `logout`, which will have the same effect)
- You can terminate programs by force by pressing Ctrl+C. If a program is waiting for an input, you can also use Ctrl+D to tell the program to stop waiting for input
  - Ctrl+D on empty command line will also exit the shell

Bérubé-Vallières © 2026
See last slide for complete credits

Wildcards

Bérubé-Vallières © 2026
See last slide for complete credits

Wildcards

- A wildcard is a symbol used to replace one or more characters in a filename. In Bash (and POSIX), these are:
- The “asterisk”: Any pattern
  - `*.doc`
- The “question mark”: Any single character
  - `?at.doc`
- The “square brackets”: Any character within the brackets
  - `cat.d[aoz]c`

Bérubé-Vallières © 2026
See last slide for complete credits

Wildcards

- A wildcard is a symbol used to replace one or more characters in a filename. In Bash (and POSIX), these are:
- The “asterisk”: Any pattern
  - `*.doc`
- The “question mark”: Any single character
  - `?at.doc`
- The “square brackets”: Any character within the brackets
  - `cat.d[aoz]c`

Inside brackets…
- Two characters separated by hyphen are a range, for example [a-d] is equivalent to [abcd]
- You can put ! at the front to match anything not in the square brackets, for example, [!a] matches any single character except the letter a

Wildcards

`echo` is useful to see how a command gets expanded!

- A wildcard is a symbol used to replace one or more characters in a filename. In Bash (and POSIX), these are:
- The “asterisk”: Any pattern
  - `*.doc`
- The “question mark”: Any single character
  - `?at.doc`
- The “square brackets”: Any character within the brackets
  - `cat.d[aoz]c`

Inside brackets…
- Two characters separated by hyphen are a range, for example [a-d] is equivalent to [abcd]
- You can put ! at the front to match anything not in the square brackets, for example, [!a] matches any single character except the letter a

Exercise

*	any pattern
?	any single char
[ ]	or

- Which files matches with the wildcards below?
  - `*.doc`
  - `?at.doc`
  - `cat.d[aoz]c`
  - `*.d[aoz]?`

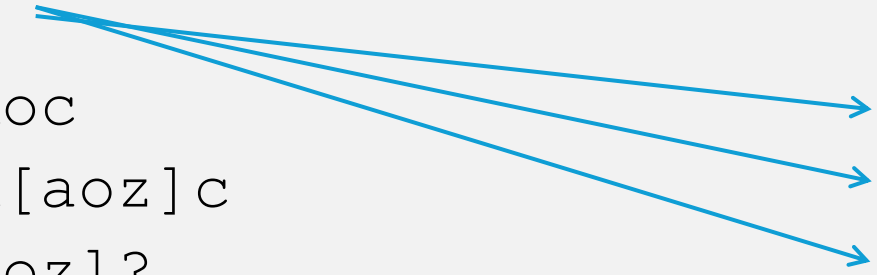
john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Bérubé-Vallières © 2026
See last slide for complete credits

Exercise

*	any pattern
?	any single char
[ ]	or

- Which files matches with the wildcards below?
  - `*.doc`
  - `?at.doc`
  - `cat.d[aoz]c`
  - `*.d[aoz]?`



john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Bérubé-Vallières © 2026
See last slide for complete credits

Exercise

*	any pattern
?	any single char
[ ]	or

- Which files matches with the wildcards below?
  - `*.doc`
  - `?at.doc`
  - `cat.d[aoz]c`
  - `*.d[aoz]?`



john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Bérubé-Vallières © 2026
See last slide for complete credits

Exercise

*	any pattern
?	any single char
[ ]	or

- Which files matches with the wildcards below?
  - `*.doc`
  - `?at.doc`
  - `cat.d[aoz]c`
  - `*.d[aoz]?`



john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Bérubé-Vallières © 2026
See last slide for complete credits

Exercise

*	any pattern
?	any single char
[ ]	or

- Which files matches with the wildcards below?
  - `*.doc`
  - `?at.doc`
  - `cat.d[aoz]c`
  - `*.d[aoz]?`



john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Bérubé-Vallières © 2026
See last slide for complete credits

Filename expansion (globbing)

- The shell recognizes and expands wildcards patterns into the list of pathnames matching the pattern. This is called *globbing*
- As the shell always does this when interpreting your input, this works for all the commands you've seen
- For example, if you type `ls *.doc` in a directory with the files on the right, you will see all the files that end with .doc files in your directory. If you type `rm ?at.doc` you will remove cat.doc and bat.doc

john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Bérubé-Vallières © 2026
See last slide for complete credits

Who globs

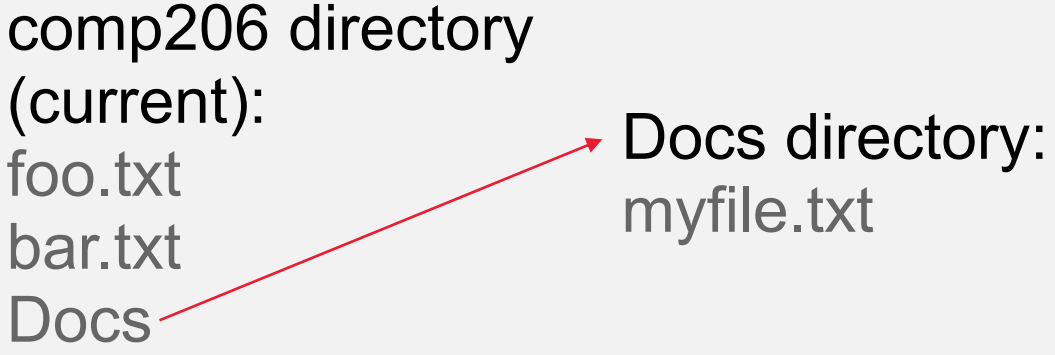
- The shell expands the wildcard patterns before giving them as arguments. For example, if you type `ls *.doc` the shell expands it into `ls john.doc cat.doc bat.doc`
  - The command has no idea that you used wildcards, the shell is the one doing the work!
- You can see this by yourself with the echo command: Try echo `ls *.doc` in a directory with doc files.

john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Bérubé-Vallières © 2026
See last slide for complete credits

Precisions on globbing

- Note that if a filename starts with a dot (i.e. is hidden), it must be matched explicitly
- For example, `*.txt` does not match `.hidden.txt`
  - But `.*.txt` does
- Globbing is applied on each segments of a pathname separately (i.e. each parts separated by /). For example, if you are in the `comp206` directory below and want to match `Docs/myfile.txt`, typing `*.txt` would not match it. However, `Docs/*.txt` or `*/*.txt` would match it.



If wildcards can’t be matched

- If the wildcards can’t be matched, the shell itself won’t do anything with them or send an error message, and the wildcards will simply be sent to the command literally
- For example, in a directory with no `.doc` files, echo `ls *.doc` will echo back `ls *.doc` and `ls` will think that you’re trying to open a file literally called `*.doc`
- This can cause unexpected results

Bérubé-Vallières © 2026
See last slide for complete credits

Command substitution

- The shell can also substitute the outputs of commands
- Syntax: `$(command)`
- Example: `touch $(date +%Y-%m-%d).txt`
  - The shell first runs `date +%Y-%m-%d`
    - `date` is a command that outputs today's date. `+%Y-%m-%d` specifies that the format should be YYYY-MM-DD
  - The output of this command today is `2026-09-10`
  - The shell replaces `$(date +%Y-%m-%d)` by `2026-09-10`
    - i.e. the command becomes `touch 2026-09-10.txt`
  - `touch` creates file `2026-09-10.txt`

Bérubé-Vallières © 2026
See last slide for complete credits

Redirection

Bérubé-Vallières © 2026
See last slide for complete credits

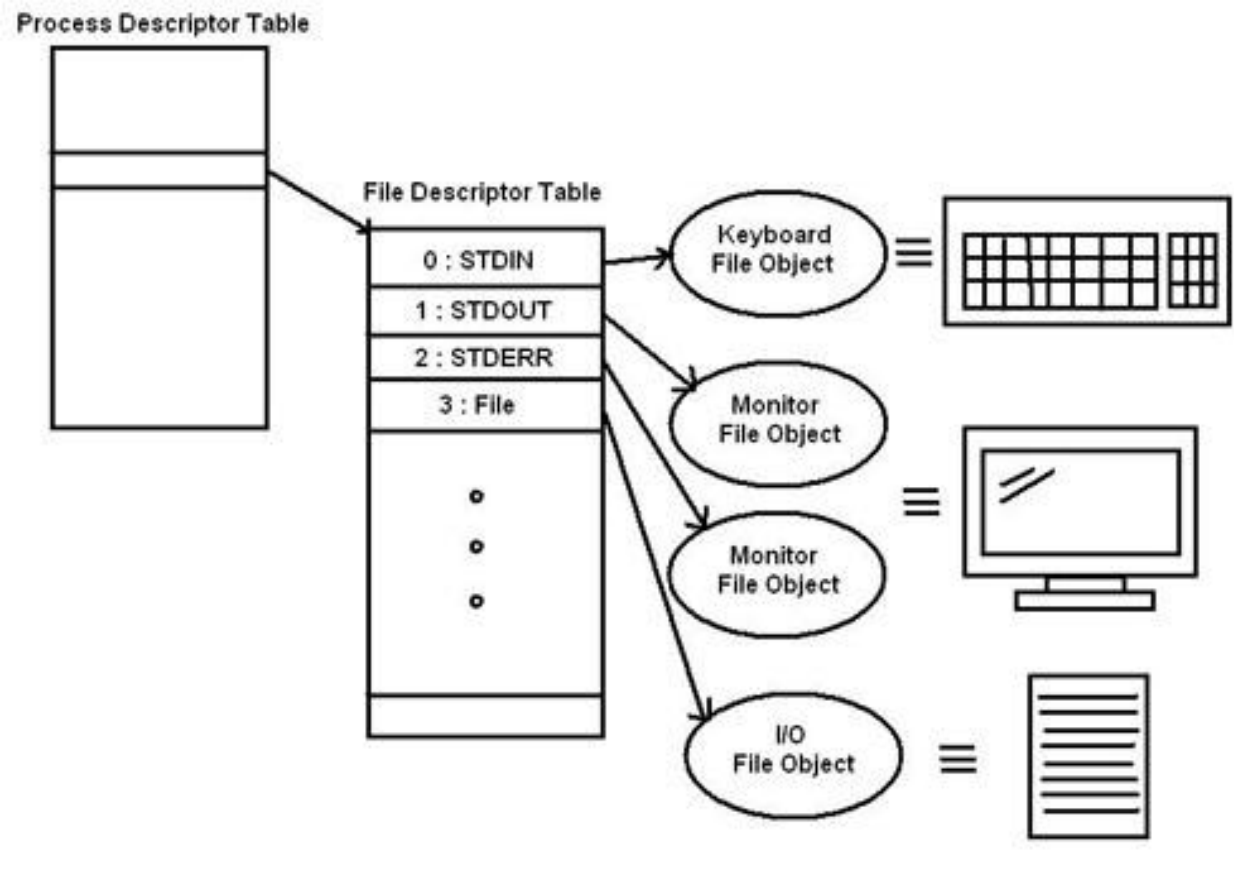
Standard streams

- Communication channels between a computer program and its environment when it begins execution
- The three standard streams are:
  - STDIN (Standard In) : this is the channel were keys typed by the user are gathered.
  - STDOUT (Standard Out) : this is the channel where normal application output is sent.
  - STDERR (Standard Error) : this is the channel where error output is sent.
- Normal output and error output is separated on two different channels since they are often monitored in different ways.

Bérubé-Vallières © 2026
See last slide for complete credits

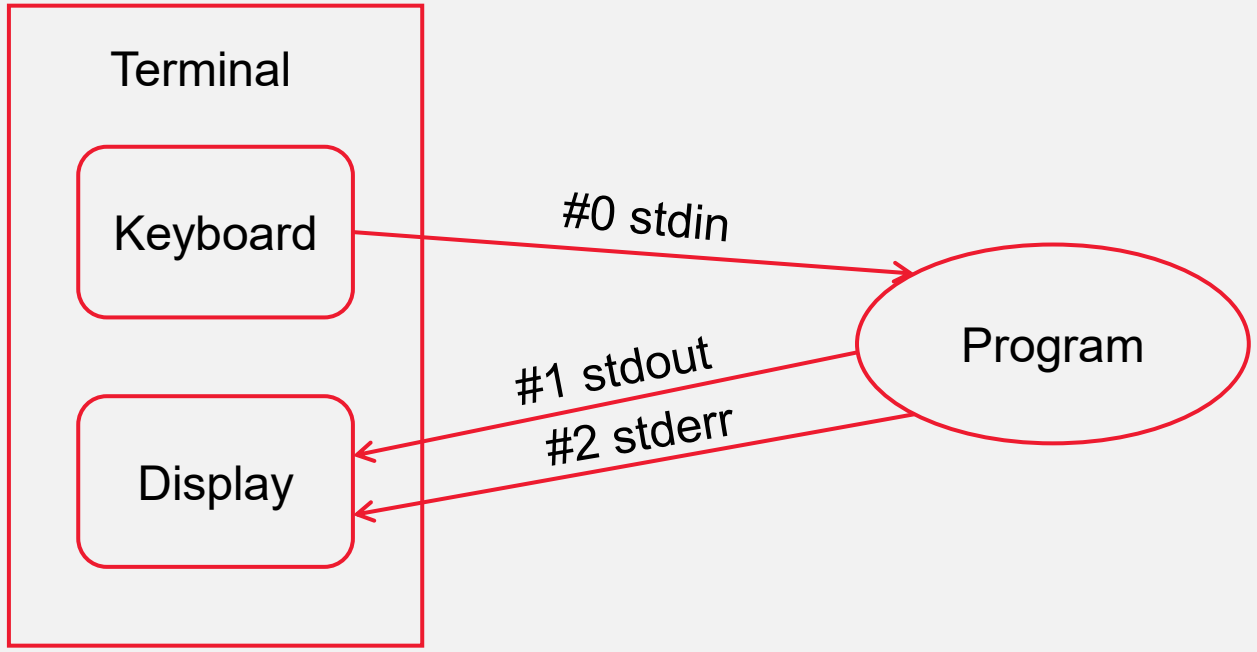
File descriptors

- When a file is opened, Unix creates a unique identifier called a file descriptor to refer to that file.
  - File descriptor table keeps tracks of all files currently open
- Unix has three special file descriptors which are always opened, and always have the same index in the file descriptor table, representing the three standard streams:
  - File descriptor 0: STDIN (Standard In)
  - File descriptor 1: STDOUT (Standard Out)
  - File descriptor 2: STDERR (Standard Error



Bérubé-Vallières © 2026
See last slide for complete credits

Programs and standard stream



Bérubé-Vallières © 2026
See last slide for complete credits

Redirection

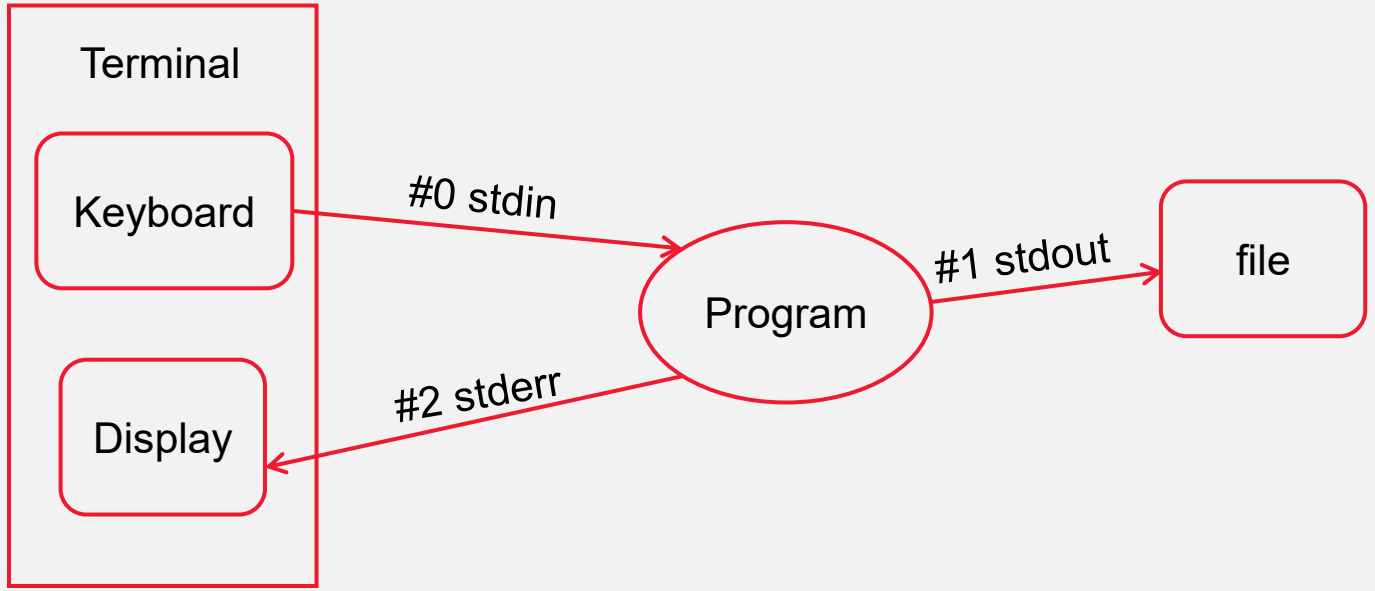
- By default:
  - stdin goes from keyboard to program
  - stdout and stderr goes from program to display
- But can redirect these streams!

Bérubé-Vallières © 2026
See last slide for complete credits

Redirecting stdout to a file

Redirect stdout to a file instead of display:

- Symbol: `>`
- Ex: `ls -la > list.txt`



Bérubé-Vallières © 2026
See last slide for complete credits

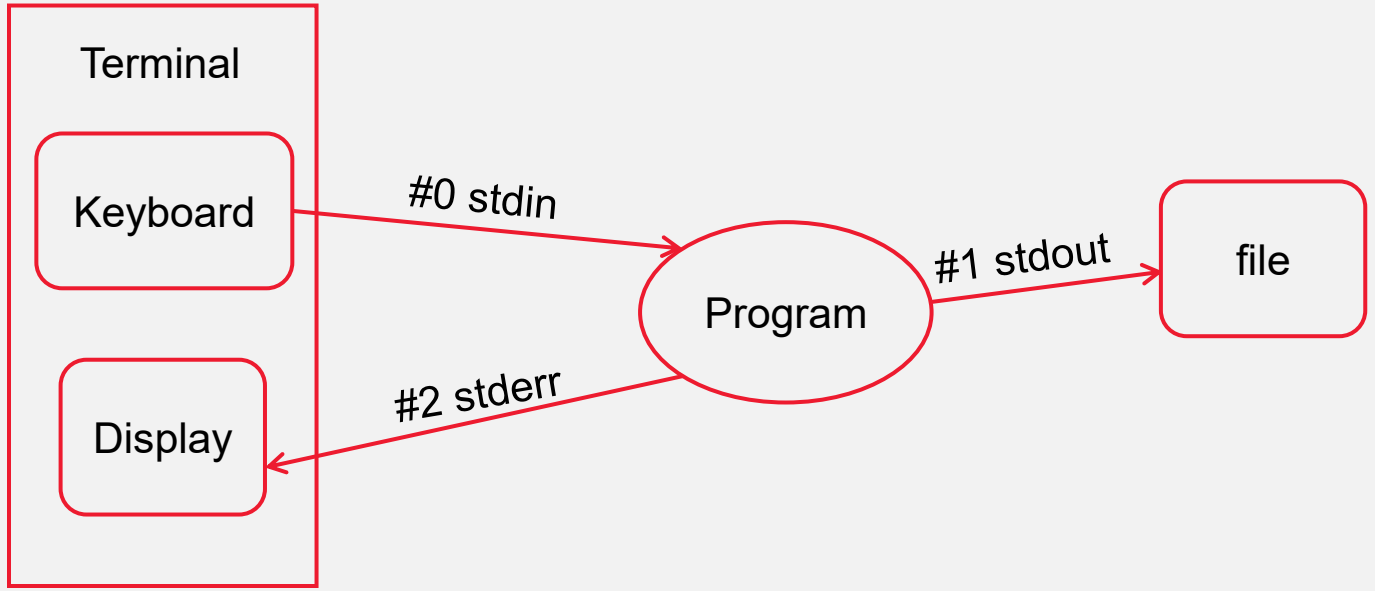
Redirecting stdout to a file

Redirect stdout to a file instead of display:

- Symbol: `>`
- Ex: `ls -la > list.txt`

Variant: append to existing file instead of overwriting

- Symbol: `>>`
- Ex: `ls -la >> list.txt`

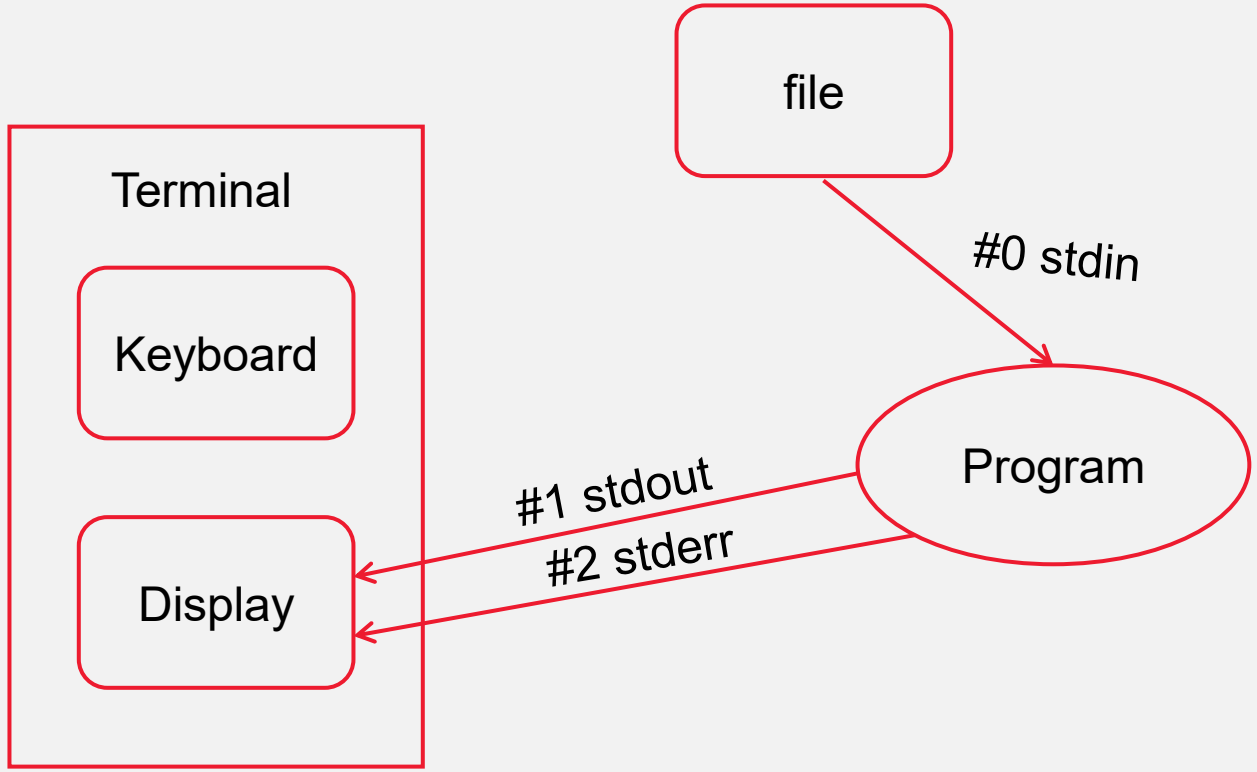


Bérubé-Vallières © 2026
See last slide for complete credits

Redirecting stdin from a file

Redirect stdin from a file instead of keyboard:

- Symbol: `<`
- Ex: `my_program < input.txt`

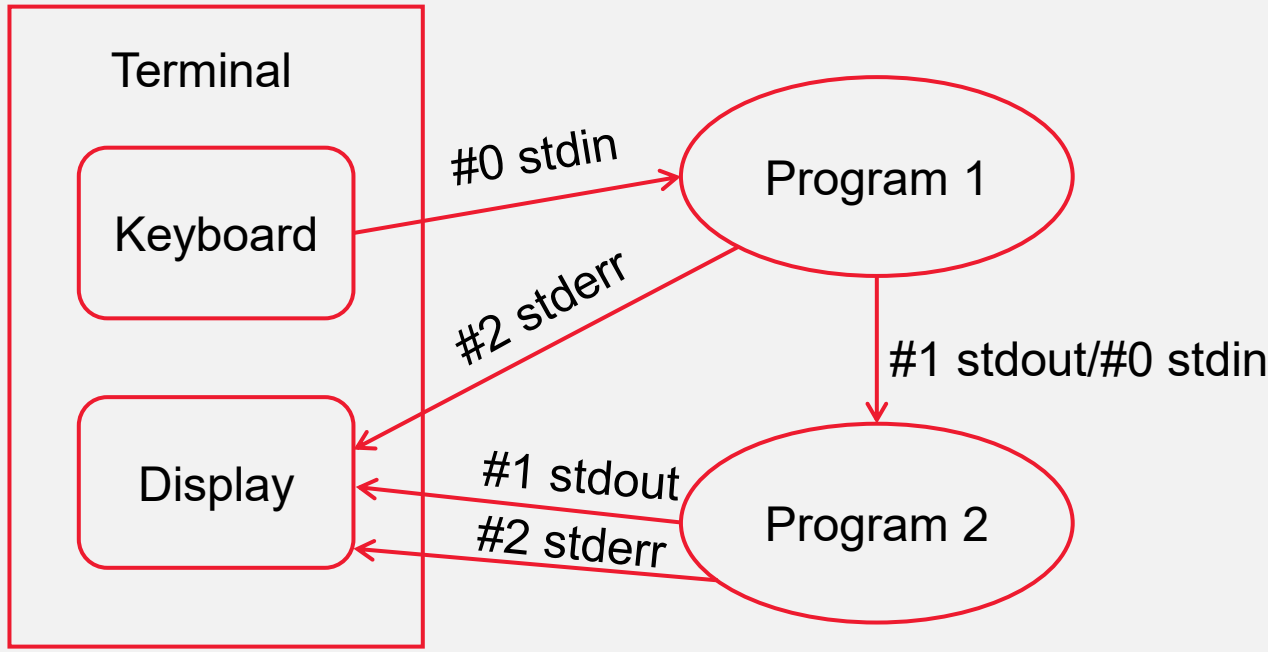


Bérubé-Vallières © 2026
See last slide for complete credits

Connecting stdin/stdout of two programs: Piping

Redirecting stdout from a program to stdin of a program has a special name: *Piping*

- Symbol: `|` (called *pipe*)
- Ex: `ls -l | more`



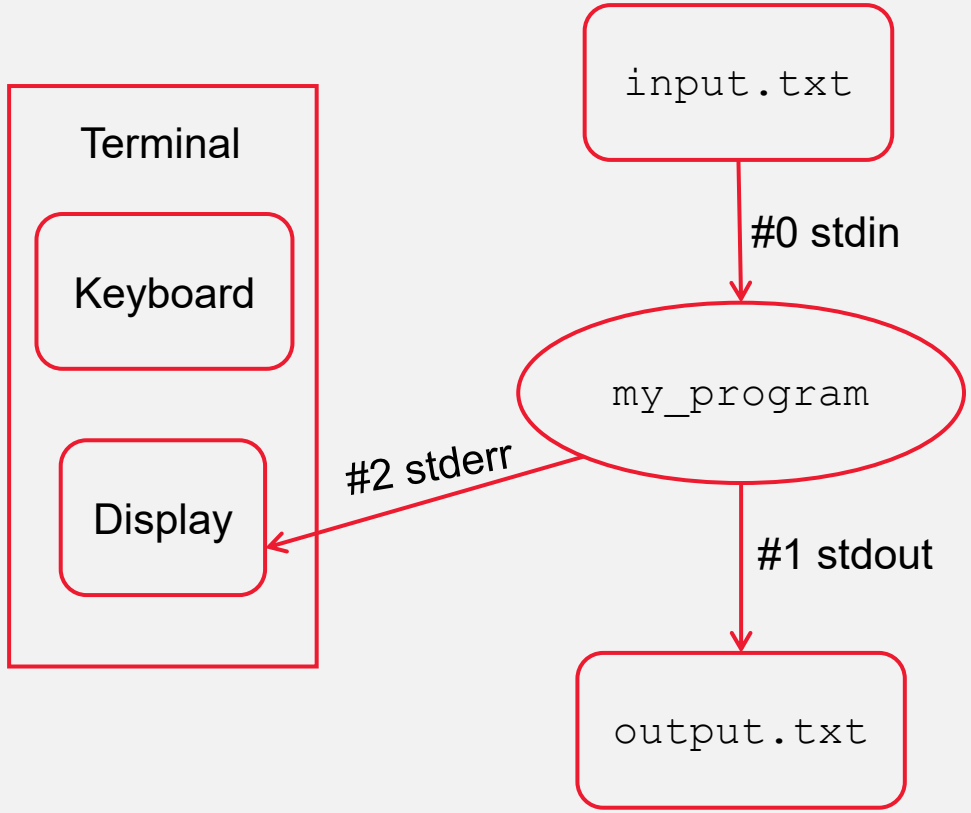
Bérubé-Vallières © 2026
See last slide for complete credits

Chaining!

Redirection and piping can be chained together to make complex commands!

- Ex: Get input from file, and output to another file

```
my_program < input.txt > output.txt
```



Bérubé-Vallières © 2026
See last slide for complete credits

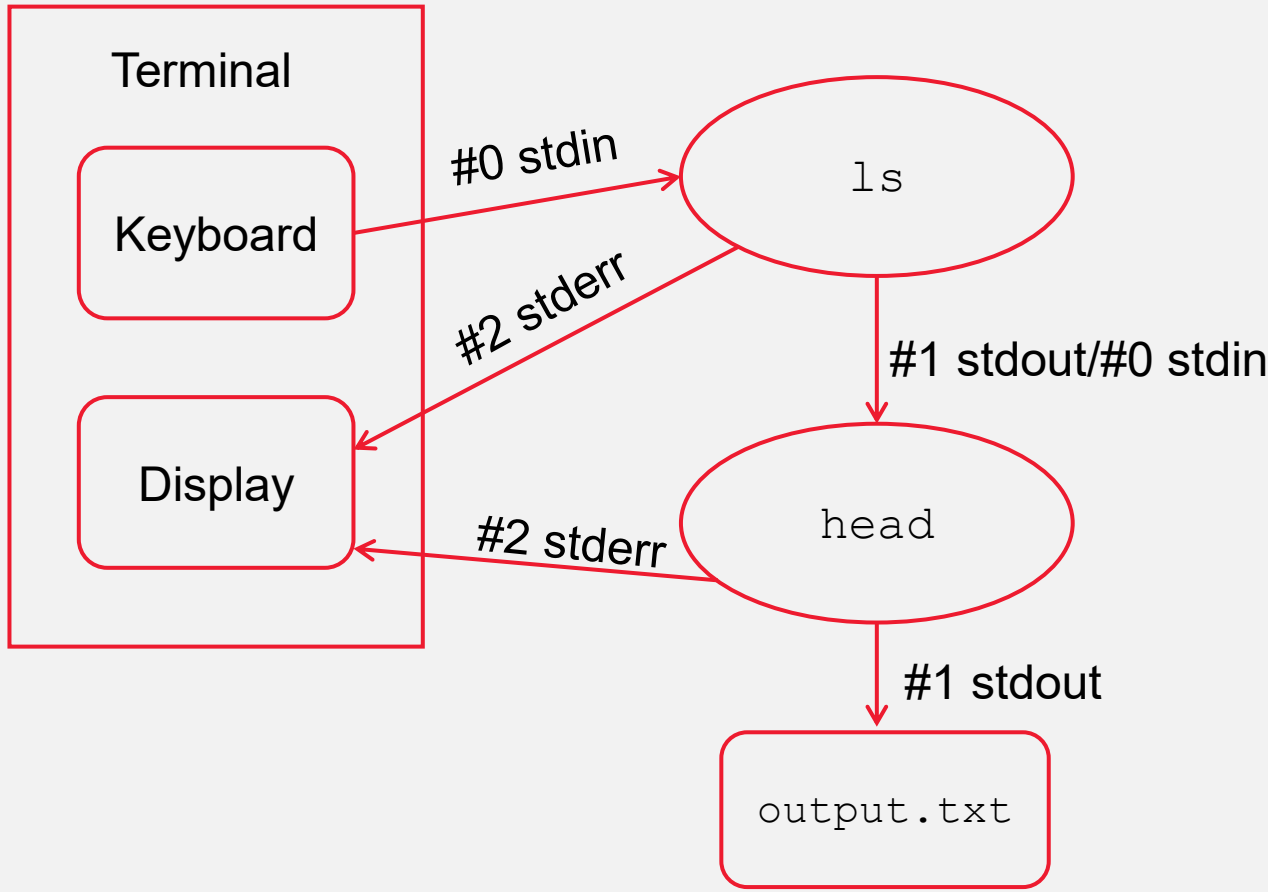
Chaining!

Redirection and piping can be chained together to make complex commands!

- Ex: Pipe and redirect output to file

```
ls -l | head > output.txt
```

(`head` shows the first 10 lines)



Bérubé-Vallières © 2026
See last slide for complete credits

Another example

- `cat $(ls *.log | tail -n5) >> text.out`
  - Assume we have files `f1.log f2.log … f20.log`

Bérubé-Vallières © 2026
See last slide for complete credits

Another example

- `cat $(ls *.log | tail -n5) >> text.out`
  - Assume we have files `f1.log f2.log … f20.log`
  - Shell interprets `ls *.log | tail -n5` first
    - Expands `ls *.log` to `f1.log f2.log … f20.log`
    - `ls` gives back the list of files
    - This is piped to `tail`
      - `tail` shows the last 10 lines by default, but the `-n5` flag here makes it show only the last 5 lines
  - Shell substitutes `$(ls *.log | tail -n5)` by the output of the command, i.e. `f16.log f17.log … f20.log`
  - cat concatenates the concents of the files (e.g. `f16.log f16.log … f19.log)` and the output is redirected to text.out

Bérubé-Vallières © 2026
See last slide for complete credits

Redirection: More variants (1/2)

(Don't memorize these)

- Redirect stderr to a file:
  - Symbol: `2>`
    - (And `2>>` to append)
  - Examples:
    - `my_program 2> error.log`
    - `my_program > output.txt 2> error.log`
- Redirect stderr to stdout:
  - Symbol: `2>&1`
  - Example: `my_program > output.txt 2>&1`

Bérubé-Vallières © 2026
See last slide for complete credits

Redirection: More variants (2/2)

(Don't memorize these)

- Can redirect to *both* stdout and a file by piping with the `tee` command
- Example:
  - `ls -l | tee output.txt`
    - Output goes to stdout and output.txt
  - `ls -l | tee -a output.txt`
    - Output goes to stdout and is appended at the end of output.txt

Bérubé-Vallières © 2026
See last slide for complete credits

Next class

- Vim and permissions

Bérubé-Vallières © 2026
See last slide for complete credits

Additional credits

- Some content from this lecture is based on:
  - Past Comp 206 slides from Prof. Vybihal

Bérubé-Vallières © 2026
See last slide for complete credits